Coordinate frames & tf2

Describe the same point. Move between frames.

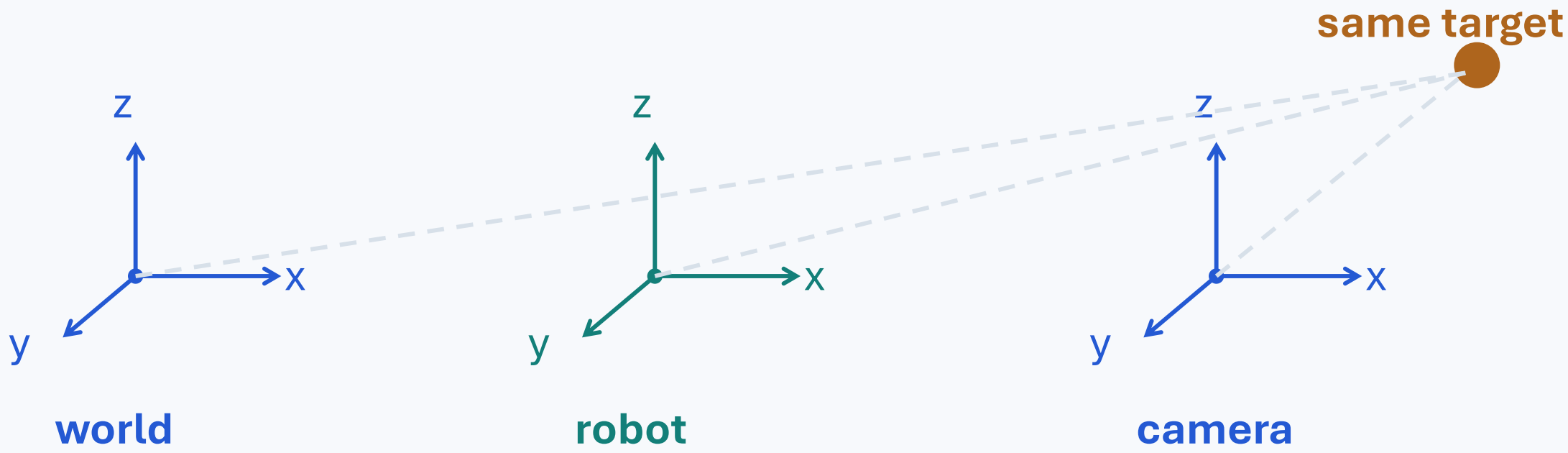
Understand frames | **Read the math** | **Inspect the demo**

$$p_A = R_{AB}\, p_B + t_{AB}$$

20 min concepts · 25 min code, demo & questions

Why do coordinates need a frame?

One physical target. Several coordinate descriptions.



A camera measurement needs a robot-frame destination.

What are TF and tf2?

A geometric relationship, managed by a ROS 2 library.

Transform (TF) · mathematics

Relationship between frames
Rotation + translation
Converts point coordinates

$$ {}^{A}p = {}^{A}R_{B}\ {}^{B}p + {}^{A}t_{B} $$

tf2 · ROS 2 library

Manages frame relationships
Static + time-dependent data
Looks up and applies transforms
Follows multi-frame chains

Frame = origin + coordinate axes.

A transform is the mathematics; tf2 manages and applies it in ROS 2.

Always check the conventions

ROS defines the standard. Always check the conventions you use.

ROS body

x forward · y left
z up

ROS optical

x right · y down
z forward



Right-handed: x × y = z

Translation: meters (m)
Rotation angles: radians (rad)

Check: handedness · transform direction · Euler axes/order · row/column vectors

Check the conventions of every data source, library, and API.

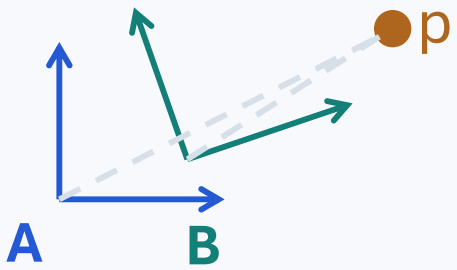
Read the direction correctly

The point stays still; its coordinate tuple changes.

$$ {}^{A}p = {}^{A}R_{B}\,{}^{B}p + {}^{A}t_{B} $$

$${}^{A}T_{B}$$

B coordinates ⟶ **A coordinates**



Pose of B in A · maps B → A

${}^{A}t_{B}$ Origin of frame B, expressed in frame A.

Elementary rotations & Euler angles

Fixed-axis XYZ: roll about x, pitch about y, yaw about z.

$R_x(\varphi)$ · roll

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\varphi & -s_\varphi \\ 0 & s_\varphi & c_\varphi \end{bmatrix}$$

$R_y(\theta)$ · pitch

$$\begin{bmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{bmatrix}$$

$R_z(\psi)$ · yaw

$$\begin{bmatrix} c_\psi & -s_\psi & 0 \\ s_\psi & c_\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R = R_z(\psi)\, R_y(\theta)\, R_x(\varphi)$$

90° x then z: $e_x \to e_y$ · z then x: $e_x \to e_z$

Pitch = ±90°: roll and yaw couple (gimbal lock).

Euler singularity; physical rotation remains possible.

▶ Gimbal Lock — Video Demonstration (start at 2:20)

Rotation matrices describe frame axes

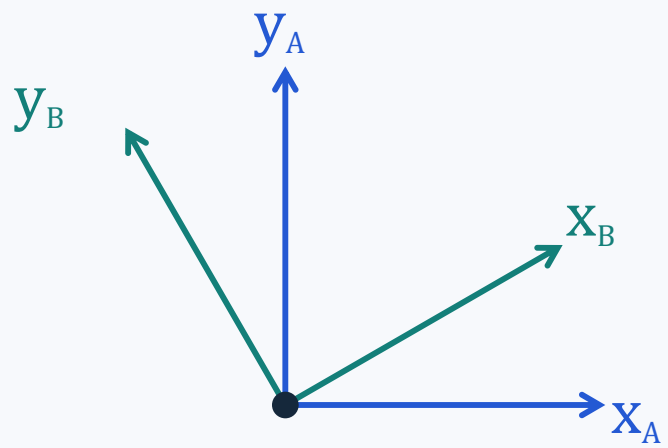
Each column is an axis of frame B expressed in frame A.

Three columns · three unit axes

$$ {}^{A}R_{B} = \begin{bmatrix} {}^{A}\hat{x}_{B} & {}^{A}\hat{y}_{B} & {}^{A}\hat{z}_{B} \end{bmatrix} $$

B axes, written using A coordinates.

$$ R^{T}R = I \qquad \det(R) = +1 $$



$\odot\ z_A = z_B$ (out)

A: blue · B: teal

Homogeneous transformations

Same 4 × 4 transformation, two notations.

Block form · 2 × 2 blocks

$$
{}^{A}T_{B} = \begin{bmatrix} {}^{A}R_{B} & {}^{A}t_{B} \\ 0^{T} & 1 \end{bmatrix}
$$

R: 3 × 3 · t: 3 × 1 · 0^{T}: 1 × 3

$$
{}^{A}p = {}^{A}R_{B}\,{}^{B}p + {}^{A}t_{B}
$$

Expanded form · 4 × 4 entries

$$
\begin{bmatrix} r_{11} & r_{12} & r_{13} & t_{x} \\ r_{21} & r_{22} & r_{23} & t_{y} \\ r_{31} & r_{32} & r_{33} & t_{z} \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

Bottom row: 0 0 0 1

Point [p; 1] includes translation.
Direction [d; 0] does not.

Compose and invert

Apply right to left. Matrix multiplication order matters.



$$ {}^{A}T_{C} = {}^{A}T_{B}\,{}^{B}T_{C} $$

Match the inner frame indices: B → B.

$$ T^{-1} = \begin{bmatrix} R^{T} & -R^{T}t \\ 0 & 1 \end{bmatrix} $$

Subtract the translation, then rotate back.

Why use quaternions?

Three practical reasons for a different rotation representation.

01 **Avoid Euler gimbal lock**

Represent every orientation without Euler-angle singularities.

02 **Compose 3D rotations**

Combine rotations using ordered quaternion multiplication.

03 **Interpolate smoothly**

SLERP gives smooth changes between orientations.

Quaternion representation

A quaternion has one scalar component and three vector components.

$$q = (w, \mathbf{v}) = (w, x, y, z)$$

w: scalar part

$\mathbf{v} = (x, y, z)$: vector part

$$w = \cos(\alpha/2)$$

$$\mathbf{v} = \mathbf{K}\sin(\alpha/2)$$

$\mathbf{K} = (k_x, k_y, k_z)$, $\|\mathbf{K}\| = 1 \cdot \alpha$: rotation angle

$$w^2 + x^2 + y^2 + z^2 = 1$$

q and −q: same 3D rotation

Quaternion multiplication & points

The Hamilton product ⊗ composes rotations. Its order matters.

$$q_1 \otimes q_2 = (w_1 w_2 - \mathbf{v}_1 \cdot \mathbf{v}_2,\ w_1 \mathbf{v}_2 + w_2 \mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2)$$

$$(0, {}^{A}p_{rot}) = q_{AB} \otimes (0, {}^{B}p) \otimes q_{AB}^{-1}$$

$${}^{A}p = {}^{A}p_{rot} + {}^{A}t_{B}$$

Unit q: inverse = conjugate · rotate first, then add translation.

How tf2 uses quaternions

The library handles the rotation algebra for you.

tf2 uses quaternions behind the scenes for rotations. Translation remains a separate 3D vector.

ROS component order

$$(x, y, z, w)$$

Identity rotation

$$(0, 0, 0, 1)$$

Unit norm is required. tf2 handles composition and inversion when you look up transforms between frames.

The frame tree

Parent → child edges describe the frame relationships.



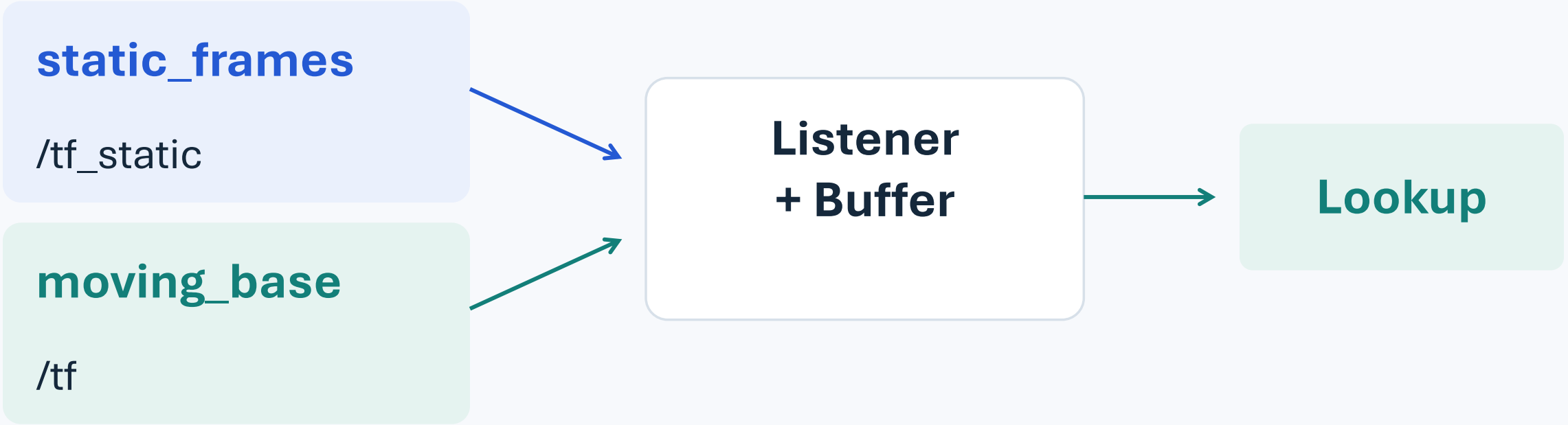
One parent per child · one authority per edge · no cycles

Disconnected trees have no transform path.

Robot joints create dynamic transforms between neighboring links.
Chain them to obtain the end-effector pose: forward kinematics.

How tf2 works

Publish edges. Buffer samples. Look up a connected path.



Several publishers · each listener has its own buffer

Transform at the measurement time

A moving robot makes an old observation different from a new one.



Finite history · consistent clocks · preserve the stamp

Inspect and predict

Predict the display before changing its reference frame.

```
ros2 run tf2_ros tf2_echo base_link camera_frame

ros2 run tf2_ros tf2_echo world base_link
```

`lookup_transform(target, source, time)`

Predict: what happens when we change the RViz Fixed Frame from base_link to world?

Live demo: read, run, predict

Three steps · predict first, then connect the output to the mathematics.



1 Inspect the transform · camera coordinates → base coordinates

```
ros2 run tf2_ros tf2_echo base_link camera_frame
```

2 Transform the point · compare with the mathematics

$${}^{C}\mathrm{p} = (0.2, 0, 1) \;\rightarrow\; {}^{B}\mathrm{p} = (0, 0.2, 1.5)$$

3 Explain a failure · missing frame or disconnected tree?